WebCar — aplikacja mobilna: podział pracy i plan Sprintu 1

Nov 3, 2026 · @Jagoda · wersja zgodna z kontekstem v2.14 i makietami z 3 XI (lista zmian na końcu)

W skrócie

Podzielcie aplikację na dwa moduły, a nie na „backend" i „UI". Każdy moduł robi jedna osoba — od wyglądu po logikę — a wspólne fundamenty mają jednego właściciela.

- **A — Jazda i łączność (propozycja: Piotrek):** moduł łączności z autem, symulator auta, ekran jazdy (01–07), sesje i statystyki.
- **B — Profile i ustawienia (propozycja: Jagoda):** wygląd (motyw, czcionki, komponenty), nawigacja, baza SQLite, ekrany 08–19.
- Przypisanie osób to tylko propozycja. Jeśli wolicie odwrotnie, zamieńcie litery — zadania się nie zmieniają.

Plan: 3 sprinty po 2 tygodnie (2 XI – 13 XII) i 3 dni buforu — aplikacja gotowa 16 XII.

Cel Sprintu 1 (2–15 XI): na iPhonie działa szkielet aplikacji. Wszystkie ekrany są w nawigacji, Home pokazuje prawdziwy stan połączenia z symulatorem auta i aktywny profil z bazy, a prototyp ekranu jazdy wysyła 20 ramek `drv` na sekundę przy sterowaniu dwoma palcami.

Założenie czasu: Sprint 1 to ok. 35 h na osobę (ok. 17 h tygodniowo). Gdy brakuje czasu, przesuwacie zadania oznaczone „Może poczekać".

Co jest w folderze i co z tego wynika

Kod aplikacji jeszcze nie istnieje, firmware ma tylko blokujący program testowy, a protokół jest szkicem. Dlatego aplikację trzeba na początku pisać pod symulator auta, a nie pod prawdziwe ESP32.

Plik	Zawartość	Znaczenie dla aplikacji
`WebCar_kontekst.md` (v2.14, 3 XI)	sprzęt, łączność (sekcja 3), zachowanie funkcji (4, 7), stack (8), protokół (9), ekrany (11), stałe (12)	główne źródło — zadania niżej odwołują się do jego sekcji
`WebCar - all screens.html`, `WebCar-2.pdf`	makiety 25 stanów ekranów (01–19, w tym 05A)	kolory, czcionki, teksty, układ; przy różnicy wygrywa opis z kontekstu. Aktualny jest plik HTML — PDF makiet traktujcie jako aktualny tylko wtedy, gdy wygenerowaliście go po ostatnich zmianach
`WebCar_plan_pracy.pdf`	starszy plan sprzętowy (lipiec)	nieistotny; ma starą mapę pinów — obowiązuje kontekst
schemat, PCB/Gerber, kosztorys	płytka główna i koszty	nieistotne dla aplikacji

Gdzie jest praca. Najwięcej trudnej logiki siedzi w ekranie jazdy i w łączności; najwięcej ekranów — w profilach i ustawieniach.

Obszar	Stany ekranów	Trudność	Co jest trudne
Ekran jazdy	01–07 (9)	wysoka	drążek i pedał naraz (multi-touch), pętla 20 Hz, model pedałów, CRUISE, E-STOP, auto-stop, CONNECTION LOST, GYRO, wideo MJPEG i CAMERA CONNECTION LOST (05A)
Łączność	— (używana wszędzie)	wysoka	wznawianie połączenia, ping co 0,5 s, wskaźnik 4 kresek, synchronizacja po połączeniu (cfg, radio, podświetlenie)
Ustawienia z autozapisem	13, 14, 18, 18A, 19 (5)	średnia	zapis i wysyłanie zmian od razu (bez SAVE), podgląd podświetlenia na Light & sound, TEST HORN i NO CONNECTION, synchronizacja radia
Profile	10–11B (5)	niska–średnia	dodawanie, zmiana nazwy, usuwanie, walidacja nazw
Home, Menu, Telemetry	08, 08A, 09, 15–17 (6)	niska	wyświetlanie danych z bazy i telemetrii

Najtrudniejsza logika jest spleciona z gestami na jednym ekranie (jazda). To przesądza o podziale pracy — niżej.

Backend/UI czy ekran po ekranie? Ani jedno, ani drugie

Rekomendacja: wariant 3 — dwa moduły z właścicielami. Czysty podział backend/UI rozcina ekran jazdy na pół, a wspólna praca ekran po ekranie daje konflikty we wspólnym kodzie.

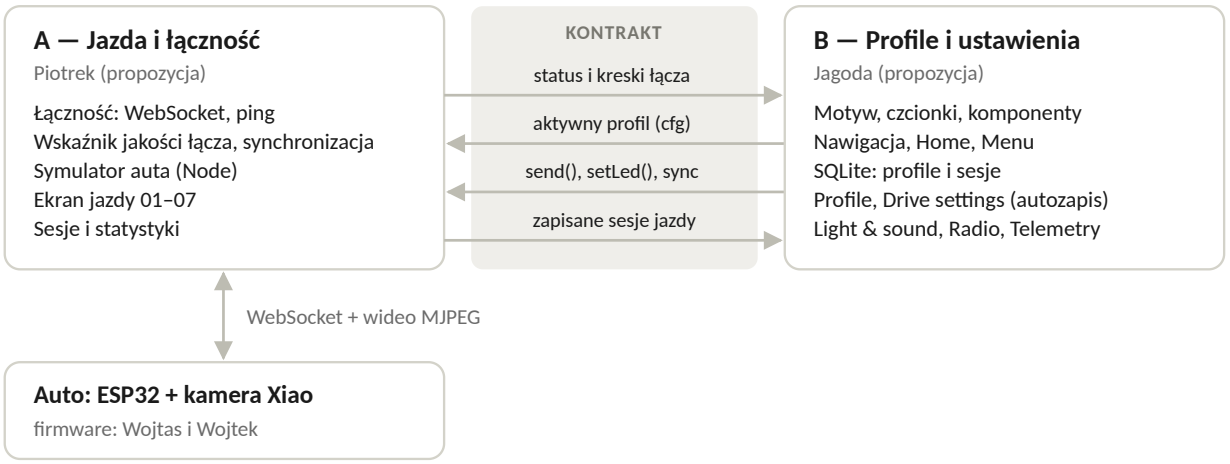
W tej aplikacji nie ma serwera. „Backend” to warstwa logiki w telefonie: WebSocket, SQLite, pętla sterowania. Jest jej mniej niż UI, ale jest najtrudniejsza i najmocniej splata się z ekranem jazdy.

Kryterium	1. Backend / UI	2. Ekran po ekranie, oboje wszystko	3. Dwa moduły (rekomendacja)
Ekran jazdy	rozcięty: gesty u jednej osoby, pętla 20 Hz u drugiej — ciągłe przekazywanie	robiony wspólnie albo przez jedną osobę bez ustaleń	w całości u A, razem z łącznością, od której zależy
Praca równoległa	tak, ale UI czeka na logikę przy każdym ekranie	trudna — oboje zmieniacie te same pliki	tak — moduły stykają się w 4 miejscach
Spójność wyglądu	dobra	słaba: dwa style, zdublowane komponenty	dobra — komponenty ma jedna osoba (B)
Obciążenie	nierówne: 25 stanów UI kontra cienka logika	równe	zbliżone: A ma mniej ekranów i trudniejszą logikę, B więcej prostszych ekranów
Ryzyko integracji	wysokie — wszystko łączy się pod koniec	średnie	niskie — każda funkcja działa od razu od ekranu do auta
Wiedza na obronę	każdy zna połowę każdego ekranu	oboje znacie wszystko, ale wolniej idzie	każdy zna swój moduł w całości, cudzy — z code review

Jak to działa w praktyce:

1. Sprint 1 to fundamenty. A buduje łączność i symulator auta, B — wygląd, nawigację i bazę. Te fundamenty są od siebie niezależne, więc nikt na nikogo nie czeka.
2. Od Sprintu 2 każdy robi ekrany swojego modułu w całości (wygląd + logika). Fundamentów drugiej osoby używa przez ustalone funkcje (kontrakt w sekcji Sprintu 1).
3. Każdy pull request przegląda druga osoba — tak oboje znacie całą aplikację przed obroną.
4. Zawór bezpieczeństwa: jeśli po Sprincie 2 ktoś jest w tyle, ekran Telemetry (15–17) przechodzi do drugiej osoby.

Każdy moduł ma jednego właściciela i styka się z drugim w 4 miejscach



podział na moduły · 4 punkty styku

Tylko A rozmawia z autem. B korzysta z łączności przez `useConnection()`, `send()` oraz dwie funkcje synchronizacji (`syncActiveProfile()`, `setLed()`), a A czyta profile przez `getActiveProfile()`.

Harmonogram: 6 tygodni

Aplikacja gotowa 16 grudnia: 3 sprinty po 2 tygodnie i 3 dni buforu

	Sprint 1 2–15 XI	Sprint 2 16–29 XI	Sprint 3 30 XI – 13 XII	Bufor 14–16 XII
A Piotrek	Łączność + symulator Wskaźnik łącza, sync cfg i LED Prototyp jazdy 20 Hz	Ekran jazdy STICK CRUISE, E-STOP, pedały CONNECTION LOST, menu	GYRO (04), kamera (05, 05A) Sesje i statystyki Tempomat z auto-stopem	Tylko poprawki **16 XII: gotowe**
B Jagoda	Motyw, czcionki, menu SQLite + profil DRIVER Home (08, 08A)	Profile (10–11B) Drive settings (13) Light & sound, Radio	Telemetry (15–17) Safe area, dopracowanie Zrzuty ekranów do pracy	
Razem + Wojtas i Wojtek	Protokół v1 do 4 XI Test hotspot + ESP32 Demo 15 XI	Jazda z apki: 29 XI (koła w powietrzu) Demo 29 XI	Testy na aucie, poprawki Koniec funkcji: 13 XII Nagranie demo	

harmonogram aplikacji · 3 sprinty + bufor

Kolejność jest celowa: najbardziej ryzykowne rzeczy (łączność, multi-touch, jazda) idą najwcześniej, a ekrany formularzowe później. Sprint 1 jest rozpisany niżej; Sprinty 2 i 3 rozpiszecie na planowaniu (W4).

Zasady pracy

Gałąź `main` zawsze działa na iPhonie i nic do niej nie trafia bez przeglądu drugiej osoby.

- **Repo:** prywatne repo `webcar-app` na GitHubie. Praca na gałęziach `a/A3-connection`, `b/B4-sqlite` (osoba + ID zadania). Pull request → review drugiej osoby w ciągu 24 h → merge.
- **Tablica:** GitHub Projects z kolumnami Do zrobienia / W toku / Review / Gotowe. Każde zadanie z tego dokumentu to jedno issue z tym samym ID.
- **Spotkania:** poniedziałek i czwartek po 15 min (zrobione / następne / blokady). Raz w tygodniu 30 min z Wojtasem i Wojtkiem (protokół, testy na aucie). Na koniec sprintu demo na iPhonie i planowanie (ok. 1,5 h).
- **Kod:** TypeScript bez przesady — wystarczą typy protokołu i profilu. Nazwy w kodzie po angielsku, formatowanie Prettier. Wszystkie stałe aplikacji z sekcji 12 kontekstu w jednym pliku `src/config.ts`.
- **Kontrakt:** pliki `src/connection/protocol.ts`, `src/connection/connection.ts`, `src/connection/sync.ts`, `src/db/profiles.ts` i oba konteksty zmienia tylko właściciel, a zmianę zatwierdza druga osoba.
- **Zmiana protokołu:** aktualizacja sekcji 9 kontekstu, podbicie wersji `fw` i wiadomość do Wojtasa i Wojtka.

Definition of Done — każde zadanie:

- [ ] działa na prawdziwym iPhonie w Expo Go, nie tylko w symulatorze iOS;
- [ ] w konsoli Metro nie ma czerwonych błędów ani żółtych ostrzeżeń z waszego kodu;
- [ ] spełnia wszystkie punkty „Gotowe, gdy” z opisu zadania;
- [ ] ekran porównany z makietą (zrzut ekranu w PR);
- [ ] PR przejrzany i zmergowany do `main`.

Sprint 1 (2–15 XI): fundamenty

Cel: szkielet aplikacji na iPhonie połączony z symulatorem auta. To zamyka największe ryzyka techniczne (łączność, multi-touch, czcionki) przed budową pełnego ekranu jazdy.

Demo 15 XI — scenariusz do odegrania na iPhonie:

1. Pierwszy start aplikacji (pusta baza) → Home: HELLO DRIVER, kafelki DRIVER, STICK, 30 % (profil startowy z sekcji 7).
2. Symulator wyłączony → CONNECTION: OFFLINE, przycisk DRIVE nieaktywny.
3. Start symulatora → w ciągu 2 s ONLINE z 4 kreskami; w konsoli symulatora pojawia się `cfg` profilu DRIVER i `act led` z `on: 0`.
4. Klawisz `p` w symulatorze (pauza telemetrii) → po 0,5 s jedna kreska, po 2 s OFFLINE. Ponownie `p` → ONLINE bez dotykania telefonu.
5. Menu → wejście na każdy ekran (na razie nagłówek i napis TODO) i powrót.
6. DRIVE → ekran poziomy. FWD jednym palcem i drążek drugim jednocześnie; symulator pokazuje ok. 20 ramek/s, rosnące `g` i zmieniające się `s`.
7. Jeśli firmware zdąży: punkty 2–4 z prawdziwym ESP32 przez hotspot iPhone'a (zadanie W3).

Struktura projektu i właściciele:

```
webcar-app/
├─ app/                     ekrany (expo-router)
│  ├─ _layout.tsx           czcionki, providery, Stack        B (+A: ConnectionProvider, start)
│  ├─ index.tsx             08 Home                           B
│  ├─ menu.tsx              09 Menu                           B
│  ├─ drive.tsx             01–07 Jazda                       A
│  ├─ profiles.tsx          10–11B Profile                    B
│  ├─ drive-settings.tsx    13                                B
│  ├─ light-sound.tsx       14 (+19)                          B
│  ├─ radio.tsx             18, 18A                           B
│  ├─ telemetry.tsx         15–17                             B
│  └─ dev.tsx               katalog komponentów (tylko dev)   B
├─ src/
│  ├─ config.ts             adresy + stałe aplikacji z sekcji 12   A
│  ├─ theme.ts              kolory, czcionki, odstępy         B
│  ├─ components/           wspólne komponenty UI             B
│  ├─ connection/           protocol, connection, linkQuality, sync   A
│  ├─ drive/                model pedałów, pętla 20 Hz        A
│  ├─ db/                   db, profiles (B), sessions (A, Sprint 3)
│  └─ state/                ConnectionContext (A), ProfileContext (B)
└─ tools/mock-car/          symulator auta (Node.js)          A
```

Kontrakt między A i B — ustalcie go w pierwszych dniach; dopóki druga część nie jest gotowa, używajcie tymczasowych stałych wartości.

```
// src/connection/connection.ts — właściciel A
export type ConnStatus = 'online' | 'offline';
export function startConnection(): void;
export function send(msg: AppToCar): boolean;          // false = brak połączenia, nic nie wysłano
export function retryNow(): void;
export function onTelemetry(cb: (t: TelMsg) => void): () => void;      // zwraca funkcję wypisania
export function onStatusChange(cb: (s: ConnStatus) => void): () => void;

// src/connection/sync.ts — właściciel A (zadanie A6)
export function syncActiveProfile(): Promise<void>;   // cfg aktywnego profilu + sprawdzenie radia
export function setLed(on: 0 | 1): void;               // stan taśmy; ponawiany po połączeniu

// src/state/ConnectionContext.tsx — A
useConnection(): { status: ConnStatus; bars: 0 | 1 | 2 | 3 | 4; pingMs: number | null;
                   telemetry: TelMsg | null; fw: string | null; send; retryNow }

// src/db/profiles.ts — właściciel B
export type Profile = { id: number; name: string; steering: 'STICK' | 'GYRO'; camera: boolean;
  blinkers: boolean; powerLimit: number; autostopCm: number; ledColor: string;
  brightness: number; horn: number; volume: number; radio: number };
export function getActiveProfile(): Promise<Profile>;
export function getProfiles(): Promise<Profile[]>;
// … reszta funkcji w zadaniu B4

// src/state/ProfileContext.tsx — B
useProfiles(): { profiles: Profile[]; activeProfile: Profile | null;
                 activate; create; rename; remove; update }
```

Zadania Sprintu 1 — suma: A ok. 34 h (+2 h opcjonalnie), B ok. 35 h (+1,5 h). Szczegóły każdego zadania w kolejnych sekcjach.

ID	Zadanie	Kto	Czas	Kiedy	Zależy od	Priorytet	Status
W1	Kickoff i setup projektu (razem)	A+B	3 h	2–3 XI	—	Musi	Do zrobienia
W2	Protokół v1 uzgodniony z Wojtasem i Wojtkiem	A+B	1,5 h	do 4 XI	—	Musi	Do zrobienia
A1	Typy protokołu i `config.ts`	A	2 h	4 XI	W1, W2	Musi	Do zrobienia
A2	Symulator auta	A	4 h	5 XI	A1	Musi	Do zrobienia
A3	Moduł łączności	A	6 h	6–8 XI	A1, A2	Musi	Do zrobienia
A4	Ping i wskaźnik jakości łącza	A	3 h	9 XI	A3	Musi	Do zrobienia
A5	`ConnectionProvider` i `useConnection`	A	2 h	10 XI	A3, A4	Musi	Do zrobienia
A6	Synchronizacja po połączeniu (profil, radio, LED)	A	2 h	11 XI	A3, B4	Musi	Do zrobienia
A7	Prototyp ekranu jazdy (multi-touch, 20 Hz)	A	7 h	11–14 XI	A3, B3	Musi	Do zrobienia
A8	Testy jednostkowe (Jest)	A	2 h	14 XI	A4, A7	Może poczekać	Do zrobienia
B1	Czcionki i test kursywy	B	3 h	3–4 XI	W1	Musi	Do zrobienia
B2	Motyw z makiet (`theme.ts`)	B	2 h	4 XI	W1	Musi	Do zrobienia
B3	Nawigacja i ekrany-zaślepki	B	3 h	5 XI	W1	Musi	Do zrobienia
B4	Baza SQLite i repozytorium profili	B	5 h	6–8 XI	W1	Musi	Do zrobienia
B5	`ProfileProvider` i `useProfiles`	B	2 h	9 XI	B4	Musi	Do zrobienia
B6	Komponenty bazowe	B	7 h	9–11 XI	B1, B2	Musi	Do zrobienia
B7	Ekran Menu (09)	B	2 h	12 XI	B3, B6	Musi	Do zrobienia

ID	Zadanie	Kto	Czas	Kiedy	Zależy od	Priorytet	Status
B8	Ekran Home (08, 08A)	B	3 h	12–13 XI	B5, B6, A5	Musi	Do zrobienia
B9	Ekran `/dev` — katalog komponentów	B	1,5 h	14 XI	B6	Może poczekać	Do zrobienia
W3	Test na sprzęcie: hotspot + ESP32 + aplikacja	A+B	2 h	12–14 XI	A3, firmware	Musi	Do zrobienia
W4	Demo, retro, plan Sprintu 2	A+B	1,5 h	15 XI	wszystkie	Musi	Do zrobienia

Sprint 1 — zadania wspólne

W1 — Kickoff i setup projektu (razem)

A + B · 3 h · 2–3 XI · przy jednym laptopie (pair programming)

Po co: jeden wspólny, działający projekt, zanim każde zacznie swoją część. Inaczej powstaną dwa różne setupy.

1. A zakłada prywatne repo `webcar-app` na GitHubie i dodaje B jako współpracownika.
2. Tworzycie projekt z domyślnego szablonu (TypeScript + expo-router) i czyścicie przykładowe ekrany. Jeśli skryptu `reset-project` nie ma w szablonie, usuńcie przykłady ręcznie.

```
npx create-expo-app@latest webcar-app
cd webcar-app
npm run reset-project
```

3. Instalujecie biblioteki ze stacku (sekcja 8 kontekstu). `npx expo install` dobiera wersje zgodne z Expo Go:

```
npx expo install expo-sqlite react-native-webview expo-sensors expo-screen-orientation \
  expo-keep-awake react-native-gesture-handler react-native-safe-area-context \
  react-native-svg expo-font expo-splash-screen
npx expo install @expo-google-fonts/saira-condensed @expo-google-fonts/jetbrains-mono
```

4. W `app.json` ustawiacie `"orientation": "default"` — pionową orientację blokuje kod, bo ekran jazdy musi móc przejść w poziom. Od razu dopisujecie wpisy z „Pułapek iOS” (sekcja 8), żeby nie zapomnieć o nich przy development buildzie:

```
"ios": {
  "supportsTablet": false,
  "infoPlist": {
    "NSLocalNetworkUsageDescription": "WebCar łączy się z pojazdem w sieci lokalnej.",
    "NSAppTransportSecurity": { "NSAllowsLocalNetworking": true }
  }
}
```

5. Tworzycie foldery ze struktury projektu (sekcja Sprint 1) z pustym plikiem `.gitkeep` w każdym — Git nie zapisuje pustych folderów.
6. Dodajecie Prettier: `npm i -D prettier` i plik `.prettierrc` z treścią `{ "singleQuote": true }`.
7. Każde uruchamia `npx expo start` i otwiera aplikację w Expo Go na iPhonie (kod QR aparatem). Jeśli telefon nie widzi laptopa: `npx expo start --tunnel`.
8. Zakładacie tablicę GitHub Projects i przepisujecie na nią zadania W1–W4, A1–A8, B1–B9.

Gotowe, gdy:

- [ ] projekt jest na `main` i oboje uruchamiacie go na iPhonie z czystego `git clone`
- [ ] `npx tsc --noEmit` przechodzi bez błędów
- [ ] tablica z zadaniami Sprintu 1 istnieje, każde zadanie ma właściciela

W2 — Protokół v1 uzgodniony z Wojtasem i Wojtkiem

A (prowadzi) + B + Wojtas i Wojtek · 1,5 h · do śr. 4 XI

Po co: sekcja 9 jest oznaczona [PROPOZYCJA]. Bez zamrożonego protokołu symulator i firmware się rozjadą, a błąd wyjdzie dopiero na aucie.

1. Przejdźcie razem każdy wiersz tabel z sekcji 9 i rozstrzygnijcie:
   - czy `hello` przychodzi od razu po połączeniu i jaka wersja `fw` (propozycja: `"1.0"`);
   - konkretna częstotliwość `tel` z zakresu 5–10 Hz (propozycja: 10 Hz);
   - format `led`: tekst `"#RRGGBB"`, 7 kolorów z makiet: `#639922`, `#2FC6DE`, `#F0B429`, `#FF4438`, `#C0308A`, `#7A3CE8`, `#F2F3F4`;
   - `d1` / `d2`: `-1` = nic w zasięgu, `null` = błąd czujnika — czy firmware umie wysłać `null`;
   - `cfg` z częścią pól zmienia tylko przysłane pola (aplikacja po połączeniu i tak wysyła wszystkie pola — sekcja 7);
   - `ping` co ok. 0,5 s na każdym ekranie to także znak obecności aplikacji: firmware gasi taśmę po zamknięciu WebSocketa albo po ok. 2 s bez żadnej wiadomości (sekcje 6 i 9);
   - drugi klient WebSocket (np. po wznowieniu połączenia) — propozycja: nowe połączenie zastępuje stare;
   - `act horn` zawsze bez numeru — gra dźwięk z `cfg.horn`, także przy TEST HORN (wybór dźwięku zapisuje się i wysyła od razu, sekcja 9).
2. Zapiszcie decyzje w sekcji 9 pliku kontekstu, usuńcie [PROPOZYCJA] i dopiszcie „Protokół v1.0 — zamrożony 4 XI 2026”.
3. Ustalcie z Wojtasem i Wojtkiem termin testu W3 (cel: 12–14 XI) i to, co firmware musi wtedy umieć (sekcja o firmware niżej).

Gotowe, gdy:

- [ ] sekcja 9 jest zaktualizowana, a Wojtas i Wojtek potwierdzili ją na czacie
- [ ] termin testu W3 jest w kalendarzu

W3 — Pierwszy test na sprzęcie: hotspot iPhone'a + ESP32 + aplikacja

A + B + Wojtas albo Wojtek · 2 h · 12–14 XI, gdy firmware ma serwer WebSocket

Po co: to pierwszy punkt z listy „Do sprawdzenia na sprzęcie” (sekcja 12). Potwierdza adresy `172.20.10.x` i daje pierwsze prawdziwe RSSI i ping.

Warunek: ESP32 łączy się z hotspotem ze stałym IP `172.20.10.13`, ma `WiFi.setSleep(false)` i serwer `/ws`, który wysyła `hello` i `tel` (wartości mogą być sztuczne) oraz odpowiada `pong`.

1. iPhone: Ustawienia → Hotspot osobisty. Nazwa i hasło jak w firmware, włączone „Pozwalaj innym na łączenie” i „Maksymalizuj zgodność”. Zostawcie otwarty ekran hotspotu.

2. Włączcie ESP32. Jeśli firmware serwuje stronę, otwórzcie `http://172.20.10.13/` w Safari na tym iPhonie.
3. Laptop łączy się z tym samym hotspotem, `npx expo start`, aplikacja w Expo Go na tym samym iPhonie.
4. W `src/config.ts` ustawcie `USE_MOCK = false`. Home ma pokazać ONLINE w ciągu 2 s.
5. Reset ESP32 przyciskiem → OFFLINE, a potem samo ONLINE, bez dotykania telefonu.
6. Zmierzcie RSSI i ping w trzech miejscach: 1 m, 5 m, inny pokój. Wpiszcie wyniki do sekcji 12.
7. Napędu nie testujcie (ESC odłączony). Można sprawdzić samo serwo skrętu z prototypu A7, jeśli firmware obsługuje już `drv`.

Gotowe, gdy:

- [ ] potwierdzone, że auto ma adres z zakresu `172.20.10.x` i aplikacja się z nim łączy
- [ ] pomiary RSSI i pingu są zapisane w kontekście
- [ ] lista problemów trafiła do Wojtasa i Wojtka

W4 — Demo, retro, plan Sprintu 2

A + B · 1,5 h · nd. 15 XI

1. Odegrajcie scenariusz demo z sekcji Sprint 1, punkt po punkcie. Co nie działa, przechodzi do Sprintu 2.
2. Retro, po 3 minuty na osobę: co pomogło, co przeszkadzało, co zmieniamy.
3. Rozpiszcie zadania Sprintu 2 w tym samym formacie co tutaj (cele w harmonogramie).

Gotowe, gdy:

- [ ] zadania Sprintu 2 są na tablicy z właścicielami i szacunkami czasu

Sprint 1 — zadania Osoby A (jazda i łączność)

A1 — Typy protokołu i config.ts

A · 2 h · 4 XI · zależy od: W1, W2

Po co: jedno miejsce z każdą wiadomością protokołu i każdą stałą aplikacji z sekcji 12. Literówka w nazwie pola wyjdzie przy kompilacji, a nie na aucie.

1. `src/connection/protocol.ts` — typy wszystkich wiadomości z sekcji 9:

```
export type DrvMsg = { t: 'drv'; g: number; s: -1 | 0 | 1; c: 0 | 1 };
export type ActMsg =
  | { t: 'act'; a: 'horn' }                    // dźwięk z cfg.horn
  | { t: 'act'; a: 'led'; on: 0 | 1 }
  | { t: 'act'; a: 'estop'; on: 0 | 1 }
  | { t: 'act'; a: 'radio'; n: number };        // 0 = stop
export type CfgMsg = { t: 'cfg'; limit?: number; stop?: number; blk?: 0 | 1;
  led?: string; bri?: number; vol?: number; horn?: number };
export type PingMsg = { t: 'ping'; id: number };
export type AppToCar = DrvMsg | ActMsg | CfgMsg | PingMsg;

export type HelloMsg = { t: 'hello'; fw: string };
export type TelMsg = { t: 'tel'; d1: number | null; d2: number | null; rssi: number;
  temp: number; es: 0 | 1; as: 0 | 1; r: number };
export type PongMsg = { t: 'pong'; id: number };
export type CarToApp = HelloMsg | TelMsg | PongMsg;
```

2. W tym samym pliku `parseCarMessage(text: string): CarToApp | null` — `JSON.parse` w `try/catch`. Dla zepsutego JSON-a albo nieznanego `t` zwraca `null` i robi `console.warn`.
3. `src/config.ts`:

```
export const USE_MOCK = true;               // false = prawdziwe auto
export const MOCK_HOST = '192.168.0.10';    // IP laptopa (macOS: ipconfig getifaddr en0)
export const CAR_WS_URL = USE_MOCK ? `ws://${MOCK_HOST}:8080/ws` : 'ws://172.20.10.13/ws';
export const CAMERA_URL = 'http://172.20.10.14:81/stream';

// Stałe z sekcji 12 [SZACUNEK] — zmieniamy tylko tutaj
export const DRV_INTERVAL_MS = 50;          // ramki drv 20 Hz
export const PING_INTERVAL_MS = 500;        // ping na każdym ekranie
export const RECONNECT_DELAY_MS = 1000;
export const TEL_STALE_MS = 500;            // > 0,5 s bez telemetrii = 1 kreska
export const TEL_LOST_MS = 2000;            // > 2 s = OFFLINE / CONNECTION LOST
export const CAMERA_RETRY_MS = 3000;        // ponowne otwarcie strumienia (05A)
export const LINK_LEVELS = [                // od najlepszego; decyduje gorszy pomiar
  { bars: 4, minRssi: -55, maxPing: 30 },
  { bars: 3, minRssi: -65, maxPing: 50 },
  { bars: 2, minRssi: -75, maxPing: 100 },
] as const;
export const ACCEL_0_100_MS = 4000;
export const DECEL_100_0_MS = 6000;
export const BRAKE_100_0_MS = 3000;
export const STICK_DEADZONE_PT = 15;
export const GYRO_ON_DEG = 15;
export const GYRO_OFF_DEG = 10;
export const ESTOP_HOLD_MS = 1000;
```

Gotowe, gdy:

- [ ] B importuje typy i stałe, `npx tsc --noEmit` przechodzi
- [ ] żadna z tych liczb nie występuje na sztywno w innym pliku

A2 — Symulator auta

A · 4 h · 5 XI · zależy od: A1

Po co: firmware ma dziś tylko program testowy. Symulator mówi tym samym protokołem co ESP32, więc całą aplikację da się zbudować i przetestować bez auta, włącznie z utratą połączenia. Przyda się też Wojtasowi i Wojtkowi jako wzorzec.

1. Folder `tools/mock-car`, w nim `npm init -y` i `npm i ws`.
2. Plik `server.js` — punkt wyjścia (rozbudowujcie wedle potrzeb):

```
// tools/mock-car/server.js — uruchom: node server.js
const { WebSocketServer } = require('ws');
const wss = new WebSocketServer({ port: 8080, path: '/ws' });
const car = { es: 0, as: 0, r: 0, led: 0, g: 0, s: 0, c: 0,
  paused: false, pongDelay: 0, weak: false, d1Error: false };
let drvCount = 0, lastDrv = 0, maxGap = 0, lastMsgAt = 0;
const sendAll = (m) => wss.clients.forEach((c) => c.readyState === 1 && c.send(JSON.stringify(m)));
const ledOff = (why) => { if (car.led) console.log('LED OFF:', why); car.led = 0; };

wss.on('connection', (ws) => {
  wss.clients.forEach((c) => { if (c !== ws) c.terminate(); }); // nowe połączenie zastępuje stare
  console.log('klient połączony, klientów:', wss.clients.size);
  ws.send(JSON.stringify({ t: 'hello', fw: 'mock-1.0' }));
  ws.on('close', () => { if (wss.clients.size === 0) ledOff('połączenie zamknięte'); });
  ws.on('message', (data) => {
    lastMsgAt = Date.now();
```

```
    let m; try { m = JSON.parse(data.toString()); } catch { return console.log('zły JSON'); }
    if (m.t === 'drv') {
      const now = Date.now();
      if (lastDrv) maxGap = Math.max(maxGap, now - lastDrv);
      lastDrv = now; drvCount++;
      if (m.g === 0) car.as = 0;                          // g = 0 zdejmuje blokadę auto-stopu
      car.g = car.es || (car.as && m.g > 0) ? 0 : m.g;   // E-STOP i blokada auto-stopu
      car.s = m.s; car.c = m.c;
    } else if (m.t === 'ping') {
      setTimeout(() => ws.send(JSON.stringify({ t: 'pong', id: m.id })), car.pongDelay);
    } else {
      if (m.t === 'act' && m.a === 'estop') car.es = m.on;
      if (m.t === 'act' && m.a === 'led') car.led = m.on;
      if (m.t === 'act' && m.a === 'radio') car.r = m.n;
      console.log('<-', JSON.stringify(m));
    }
  });
});

setInterval(() => {                                   // telemetria 10 Hz
  if (car.paused) return;
  sendAll({ t: 'tel', d1: car.d1Error ? null : 40 + Math.round(Math.random() * 100),
    d2: Math.random() < 0.5 ? -1 : 30 + Math.round(Math.random() * 50),
    rssi: car.weak ? -80 : -50 - Math.round(Math.random() * 4), temp: 47,
    es: car.es, as: car.as, r: car.r });
}, 100);

setInterval(() => {                                   // kontrola raz na sekundę
  if (lastDrv && Date.now() - lastDrv > 300) {
    console.log('FAILSAFE: brak drv > 300 ms → moc 0, koła prosto');
    car.g = 0; car.s = 0; lastDrv = 0;
  }
  if (lastMsgAt && Date.now() - lastMsgAt > 2000) ledOff('brak wiadomości > 2 s');
  if (drvCount) {
    console.log(`drv ${drvCount}/s, max przerwa ${maxGap} ms, g=${car.g} s=${car.s} c=${car.c}`);
  }
  drvCount = 0; maxGap = 0;
}, 1000);

process.stdin.setRawMode(true); process.stdin.resume(); process.stdin.setEncoding('utf8');
process.stdin.on('data', (k) => {                     // klawisze scenariuszy testowych
  if (k === 'p') car.paused = !car.paused;
  if (k === 'k') wss.clients.forEach((c) => c.terminate());
  if (k === 'l') car.pongDelay = car.pongDelay ? 0 : 120;
  if (k === 'w') car.weak = !car.weak;
  if (k === 'e') car.d1Error = !car.d1Error;
  if (k === 'a' && car.c) { car.as = 1; car.g = 0; }
  if (k === 'r') car.r = car.r ? 0 : 3;
  if (k === 'q' || k === '\u0003') process.exit();
  console.log('stan:', JSON.stringify(car));
});
```

3. Klawisze: `p` pauza telemetrii (utrata łącza), `k` zerwanie połączenia, `l` opóźnienie `pong` +120 ms, `w` słaby sygnał (RSSI −80), `e` błąd przedniego czujnika, `a` auto-stop (tylko przy `c` = 1; `as` = 1 aż do ramki `drv` z `g` = 0, jak w sekcji 4), `r` radio gra / nie gra (test synchronizacji radia w A6), `q` wyjście. Opiszcie je w `tools/mock-car/README.md`.
4. Symulator, tak jak firmware (sekcja 6), gasi podświetlenie po zamknięciu połączenia albo po 2 s bez żadnej wiadomości od aplikacji — w konsoli widać wtedy `LED OFF`.
5. Test bez aplikacji: `npx wscat -c ws://localhost:8080/ws`, wyślij `{"t":"ping","id":1}` → ma wrócić `pong`.

Gotowe, gdy:

- [ ] test z `wscat` przechodzi i widać ramki `tel` co 100 ms

- [ ] README opisuje uruchomienie i klawisze

A3 — Moduł łączności

A · 6 h · 6–8 XI · zależy od: A1, A2

Po co: jedno miejsce, które utrzymuje połączenie z autem (sekcja 3: `send`, `onTelemetry`, `onStatusChange`). Reszta aplikacji nie zna szczegółów WebSocketa.

1. `src/connection/connection.ts` jako zwykły moduł, nie komponent. Zmienne na poziomie modułu: `ws`, `status`, `lastTelAt`, `reconnectTimer`, zbiory słuchaczy (`Set`).
2. `startConnection()` wołana raz w `app/_layout.tsx`. Flaga `started` chroni przed podwójnym startem po Fast Refresh.
3. `connect()`: `const socket = new WebSocket(CAR_WS_URL); ws = socket;`. W `onmessage` — `parseCarMessage`; dla `tel` ustaw `lastTelAt = Date.now()` i wywołaj słuchaczy; dla `hello` zapamiętaj `fw`.
4. `onclose` i `onerror`: najpierw `if (socket !== ws) return;` (zdarzenia starego gniazda ignorujemy), potem `ws = null` i `scheduleReconnect()`.
5. `scheduleReconnect()` — tylko jeden timer naraz: jeśli `reconnectTimer` istnieje, nic nie rób; inaczej `setTimeout(connect, RECONNECT_DELAY_MS)`.
6. Status: `setInterval` co 250 ms. `ws` otwarte i `Date.now() - lastTelAt < TEL_LOST_MS` → `online`, inaczej `offline`. Słuchaczy `onStatusChange` wołaj tylko przy zmianie.
7. `retryNow()`: wyczyść timer, odepnij handlery starego gniazda (`old.onclose = null` itd.), `old.close()`, `connect()`.
8. `send(msg)`: gdy `ws?.readyState === WebSocket.OPEN` → `ws.send(JSON.stringify(msg))` i `true`; inaczej `false`.
9. `onTelemetry` i `onStatusChange` dodają funkcję do zbioru i zwracają funkcję, która ją usuwa.
10. `AppState` z `react-native`: przejście w tło → zamknij gniazdo tak jak w `retryNow` (odpięte handlery, wyczyszczony timer), ale bez ponownego łączenia, i zatrzymaj ping. Failsafe zatrzyma auto, a firmware zgasi taśmę (sekcja 6) — tak ma być. Powrót → `connect()`.
11. Logi z prefiksem `[conn]`: open, close, hello fw=…, reconnect.

Gotowe, gdy (test z symulatorem):

- [ ] start aplikacji przy działającym symulatorze → `online` w mniej niż 2 s
- [ ] klawisz `k` → aplikacja łączy się ponownie w ok. 1–2 s
- [ ] symulator zatrzymany (Ctrl+C) → `offline` po 2 s, próby co 1 s bez zalewania logów; ponowny start symulatora → `online` samo
- [ ] po kilku Fast Refresh symulator pokazuje 1 klienta
- [ ] aplikacja w tle → `FAILSAFE` i `LED OFF` w symulatorze (jeśli taśma była włączona); powrót → `online`

A4 — Ping i wskaźnik jakości łącza

A · 3 h · 9 XI · zależy od: A3

Po co: cztery kreski na Home i na ekranie jazdy (sekcja 3) oraz ping w ms. Ping jest też dla firmware znakiem, że aplikacja jest połączona (gaszenie taśmy, sekcja 6).

1. W module łączności co `PING_INTERVAL_MS` (0,5 s), na każdym ekranie, gdy gniazdo jest otwarte, wysyłaj `{ t: 'ping', id }` (rosnące `id`) i zapisz czas wysłania w `Map`.
2. Przy `pong`: `rtt = Date.now() - sentAt`. Trzymaj 3 ostatnie RTT; `pingAvg` = ich średnia. Usuwaj z mapy pingi starsze niż 3 s.
3. Gdy przez 3 s nie przyszedł żaden `pong`, `pingAvg = null` — UI pokaże „— MS".
4. `src/connection/linkQuality.ts` — czysta funkcja bez Reacta:

```
export function computeLinkBars(rssi: number | null, pingAvg: number | null,
                                msSinceTel: number): 0 | 1 | 2 | 3 | 4 {
  if (msSinceTel > TEL_LOST_MS) return 0;                  // CONNECTION LOST: kreski zgaszone
  if (msSinceTel > TEL_STALE_MS || rssi === null) return 1;
  for (const l of LINK_LEVELS) {
    if (rssi >= l.minRssi && (pingAvg === null || pingAvg <= l.maxPing)) return l.bars;
  }
  return 1;
}
```

5. Obok: `pingColor(bars)` — 4 i 3 kreski: biały, 2: żółty (accent), 1: czerwony (danger).

Oczekiwane wyniki:

RSSI (dBm)	Ping, średnia z 3 (ms)	Bez telemetrii (s)	Kreski	Dlaczego
−50	20	0,1	4	spełnia progi 4 kresek
−50	45	0,1	3	decyduje ping
−70	20	0,1	2	decyduje RSSI
−80	20	0,1	1	RSSI poniżej −75
−50	120	0,1	1	ping powyżej 100
−50	20	0,7	1	ponad 0,5 s bez telemetrii
−50	20	2,5	0	CONNECTION LOST
−60	brak	0,1	3	bez pingu liczy się samo RSSI

Gotowe, gdy:

- [ ] wszystkie przypadki z tabeli dają oczekiwany wynik (testy w A8 albo ręcznie)
- [ ] symulator, klawisz `l` → po ok. 1,5–2 s jedna kreska (średnia z 3 pingów co 0,5 s); ponownie `l` → wraca do 4
- [ ] klawisz `w` → jedna kreska

A5 — ConnectionProvider i useConnection

A · 2 h · 10 XI · zależy od: A3, A4

Po co: ekrany B dostają stan połączenia jednym hookiem, bez przerysowywania ekranu przy każdej ramce telemetrii (10 razy na sekundę).

1. `src/state/ConnectionContext.tsx`: provider subskrybuje moduł; ostatnia telemetria w `useRef`.
2. `setInterval` co 250 ms liczy `{ status, bars, pingMs, telemetry, fw }` (jak w kontrakcie) i robi `setState` tylko przy zmianie — najwyżej 4 odświeżenia na sekundę.
3. `useConnection()` zwraca ten stan oraz `send` i `retryNow`.
4. Provider wstawiasz do `app/_layout.tsx` — to plik B, więc uzgodnij zmianę.
5. Na Sprint 2: ekran jazdy reaguje na `as` i `es` natychmiast, więc subskrybuje `onTelemetry` bezpośrednio, nie przez kontekst.

Gotowe, gdy:

- [ ] B używa `useConnection()` na Home
- [ ] `console.count` na Home pokazuje najwyżej ok. 4 rendery na sekundę

A6 — Synchronizacja po połączeniu: profil, radio, podświetlenie

A · 2 h · 11 XI · zależy od: A3, B4

Po co: sekcja 7 — po każdym połączeniu auto dostaje ustawienia aktywnego profilu i właściwy stan podświetlenia. Źródłem prawdy jest zawsze profil w telefonie.

1. `src/connection/sync.ts`: `profileToCfg(p: Profile): CfgMsg` — `limit` ← `powerLimit`, `stop` ← `autostopCm`, `blk` ← `blinkers ? 1 : 0`, `led` ← `ledColor`, `bri` ← `brightness`, `vol` ← `volume`, `horn` ← `horn`.
2. `syncActiveProfile()`: `const p = await getActiveProfile(); send(profileToCfg(p));` (zawsze wszystkie pola), zapamiętaj `p` i ustaw flagę `radioCheck = true`.
3. Przy pierwszej ramce `tel` po ustawieniu flagi: `radioCheck = false`; jeśli `tel.r !== p.radio`, wyślij `{ t: 'act', a: 'radio', n: p.radio }`. Przy równych wartościach nic — utwór, który już gra, nie zaczyna się od nowa.
4. `setLed(on)`: zapamiętaj `ledOn = on` i wyślij `{ t: 'act', a: 'led', on }`. Bez połączenia `send` zwraca `false` — nic nie szkodzi, stan pójdzie przy synchronizacji. Na start `ledOn = 0`, bo poza ekranem jazdy i Light & sound taśma ma być zgaszona (sekcja 7). W Sprincie 2 wołają ją przycisk LED i koniec sesji (A) oraz wejście i wyjście z ekranu Light & sound (B).
5. `startSync()` wołana raz obok `startConnection()` (ta sama ochrona przed Fast Refresh): subskrybuje `onStatusChange` — przy każdym przejściu na `online` wywołuje `syncActiveProfile()` i wysyła `act led` z bieżącym `ledOn`; subskrybuje `onTelemetry` dla kroku 3.
6. B wywołuje `syncActiveProfile()` po aktywacji profilu (B5), a w Sprincie 2 także po każdej zmianie aktywnego profilu na ekranach ustawień (autozapis, sekcja 7). Ta sama funkcja wysyła wtedy `cfg` i, gdy trzeba, `act radio`.
7. Dopóki B4 nie jest gotowe, użyj profilu DRIVER z wartościami domyślnymi wpisanego na sztywno.

Gotowe, gdy:

- [ ] `cfg` ze wszystkimi polami aktywnego profilu i `act led` z `on: 0` pojawiają się w symulatorze w ciągu 1 s po każdym połączeniu, także po klawiszu `k`
- [ ] radio: klawisz `r` w symulatorze (gra utwór 3), potem `k` → aplikacja wysyła `act radio` z `n: 0` (DRIVER ma radio OFF); po kolejnym `k` (`r` już 0) nic nie wysyła

A7 — Prototyp ekranu jazdy: multi-touch, 20 Hz, orientacja

A · 7 h · 11–14 XI · zależy od: A3, B3

Po co: sprawdzić teraz trzy najbardziej ryzykowne rzeczy w całej aplikacji — dwa palce naraz, stabilne 20 Hz i blokadę orientacji. W Sprincie 2 tylko dobudowujesz funkcje. Wygląd jest tu nieistotny.

1. `app/drive.tsx`. W `_layout` dla tej trasy `gestureEnabled: false` — gest „wstecz” nie może przypadkiem zakończyć sesji.
2. Orientacja i wygaszanie ekranu:

```
useKeepAwake();
useFocusEffect(useCallback(() => {
  ScreenOrientation.lockAsync(ScreenOrientation.OrientationLock.LANDSCAPE_RIGHT);
  return () => {
    ScreenOrientation.lockAsync(ScreenOrientation.OrientationLock.PORTRAIT_UP);
  };
}, []));
```

3. Model pedałów — tabela z sekcji 4 w jednej czystej funkcji (`src/drive/pedals.ts`):

```
export type Pedal = 'FWD' | 'REV' | null;
const ACC = (100 * DRV_INTERVAL_MS) / ACCEL_0_100_MS;   // ok. 1,25 % na ramkę
const DEC = (100 * DRV_INTERVAL_MS) / DECEL_100_0_MS;   // ok. 0,83 %
const BRK = (100 * DRV_INTERVAL_MS) / BRAKE_100_0_MS;   // ok. 1,67 %

export function nextPower(p: number, pedal: Pedal): number {
  if (pedal === 'FWD') p = p >= 0 ? p + ACC : p + BRK;      // jazda wstecz + FWD = hamowanie
  else if (pedal === 'REV') p = p <= 0 ? p - ACC : p - BRK; // jazda do przodu + REV = hamowanie
  else p = p > 0 ? Math.max(0, p - DEC) : Math.min(0, p + DEC);
  return Math.max(-100, Math.min(100, p));
}
```

4. Pętla 20 Hz — bieżące wartości w `useRef` , żeby `setInterval` nie musiał się restartować:

```
const pedalRef = useRef<Pedal>(null);
const steerRef = useRef<-1 | 0 | 1>(0);
const powerRef = useRef(0);

useEffect(() => {
  const id = setInterval(() => {
    powerRef.current = nextPower(powerRef.current, pedalRef.current);
    send({ t: 'drv', g: Math.round(powerRef.current), s: steerRef.current, c: 0 });
    setShownPower(Math.round(powerRef.current));
  }, DRV_INTERVAL_MS);
  return () => {
    clearInterval(id);
    send({ t: 'drv', g: 0, s: 0, c: 0 });   // koniec sesji: nie polegamy tylko na failsafe
  };
}, []);
```

5. Gesty — drążek i każdy pedał w osobnym `GestureDetector`, dzięki czemu dwa palce działają niezależnie. `.runOnJS(true)` pozwala wołać zwykłe funkcje i `setState`:

```
const stick = Gesture.Pan().runOnJS(true).minDistance(0)
  .onBegin((e) => updateStick(e.x)).onUpdate((e) => updateStick(e.x))
  .onFinalize(() => resetStick());
const fwd = Gesture.Pan().runOnJS(true).minDistance(0)
  .onBegin(() => pressPedal('FWD')).onFinalize(() => releasePedal('FWD'));
// <GestureDetector gesture={stick}><View style={styles.stick}>…</View></GestureDetector>
```

6. `updateStick(x)`: `dx = x - promień` (pozycja palca w widoku drążka minus środek — nie `translationX`). `|dx| < STICK_DEADZONE_PT` → skręt 0, inaczej −1 albo 1. Ruch w pionie ignorujesz.
7. Tylko jeden pedał naraz: `pressPedal` ustawia `pedalRef` tylko, gdy jest `null`; `releasePedal` czyści go tylko, gdy to ten sam pedał.
8. Na ekranie, bez stylowania: kółko drążka z gałką, dwa prostokąty REV i FWD, duży napis z mocą w % i skrętem.
9. Wejście przyciskiem DRIVE z Home (B8); do tego czasu tymczasowy przycisk.
10. Na Sprint 2: przy końcu sesji aplikacja wysyła też `setLed(0)` i wyłącza tempomat, a po wejściu na ekran — stan domyślny (sekcje 3 i 7).

Gotowe, gdy:

- [ ] trzymam FWD jednym palcem i ruszam drążkiem drugim — symulator pokazuje jednocześnie rosnące `g` i zmieniające się `s`
- [ ] symulator: ok. 20 ramek/s i maks. przerwa poniżej 120 ms, także przy ruszaniu palcami
- [ ] FWD dochodzi do 100 % w ok. 4 s; po puszczeniu moc spada powoli; REV w trakcie jazdy do przodu hamuje, a potem rusza wstecz
- [ ] obrót telefonu o 180° nie zmienia orientacji; po wyjściu ekran wraca do pionu
- [ ] ekran nie wygasa przez 2 min; wyjście z ekranu → symulator dostaje `g=0 s=0`; aplikacja w tle → `FAILSAFE`

Plan B, gdyby dwa `GestureDetector` nie działały jednocześnie: jeden `Gesture.Manual()` na całym dolnym pasku ekranu i rozpoznawanie palców po pozycji (`onTouchesDown`, `onTouchesUp`). Rozstrzygnijcie to w tym sprincie.

A8 — Testy jednostkowe (może poczekać)

A · 2 h · 14 XI · zależy od: A4, A7

Po co: model pedałów i wskaźnik łącza to czyste funkcje — łatwo je przetestować, a testy przydadzą się w rozdziale pracy o weryfikacji.

1. `npx expo install jest-expo jest @types/jest -- --save-dev`; w `package.json`: `"test": "jest"` w `scripts` i `"jest": { "preset": "jest-expo" }`. `@types/jest` jest potrzebny, żeby `npx tsc --noEmit` znał `test` i `expect`.
2. `src/connection/linkQuality.test.ts` — wszystkie przypadki z tabeli w A4.
3. `src/drive/pedals.test.ts` — co najmniej: `nextPower(0, 'FWD') > 0`; `nextPower(0, 'REV') < 0`; `nextPower(100, 'FWD') === 100`; `nextPower(0.5, null) === 0` (zwalnianie nie przechodzi przez zero); hamowanie szybsze niż zwalnianie: `50 - nextPower(50, 'REV') > 50 - nextPower(50, null)`.

4. `npm test` przed każdym PR, który zmienia te pliki.

Gotowe, gdy:

- [ ] `npm test` przechodzi

Sprint 1 — zadania Osoby B (wygląd, profile, ustawienia)

B1 — Czcionki i test kursywy

B · 3 h · 3–4 XI · zależy od: W1

Po co: makiety stoją na dwóch czcionkach, a sekcja 11 każe na samym początku sprawdzić, czy iOS pochyli Saira Condensed (nie ma ona wersji kursywnej).

1. Ładowanie czcionek w `app/_layout.tsx`:

```
import { useFonts, SairaCondensed_700Bold } from '@expo-google-fonts/saira-condensed';
import { JetBrainsMono_500Medium, JetBrainsMono_700Bold } from '@expo-google-fonts/jetbrains-mono';
import * as SplashScreen from 'expo-splash-screen';

SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
  const [loaded] = useFonts({
    SairaCondensed_700Bold, JetBrainsMono_500Medium, JetBrainsMono_700Bold,
  });
  useEffect(() => { if (loaded) SplashScreen.hideAsync(); }, [loaded]);
  if (!loaded) return null;
  return /* providery + <Stack /> (B3) */;
}
```

2. Test na iPhonie: dwa napisy WEBCAR w rozmiarze 34 — z `fontStyle: 'italic'` i bez. Zróbcie zrzut ekranu.
3. Jeśli oba są proste: pochylenie przez `transform: [{ skewX: '-10deg' }]` plus `paddingRight: 4`, żeby koniec napisu nie był ucięty.
4. `src/components/Typography.tsx`:
   - `Heading` — Saira, wielkie litery, rozmiary z makiet, m.in.: 88 (Home), 44 (DRIVE), 34 (tytuły), 32 (okna), 28 (przyciski), 26 (wartości w kafelkach), 24, 20. Decyzja o kursywie zapada tylko tutaj.
   - `Label` — JetBrains Mono 700, 12–13, `letterSpacing` ok. 2, wielkie litery, kolor `muted`.
   - `Mono` — JetBrains Mono 500 (dane, liczby).

Gotowe, gdy:

- [ ] w PR jest zrzut: nagłówek pochylony jak na makietach
- [ ] decyzja (kursywa czy `skewX`) zapisana w README
- [ ] `fontFamily` nie występuje nigdzie poza `Typography.tsx` i `theme.ts`

B2 — Motyw z makiet (theme.ts)

B · 2 h · 4 XI · zależy od: W1

Po co: wszystkie kolory, czcionki i odstępy w jednym pliku. Wartości poniżej pochodzą z aktualnego kodu makiet (`WebCar - all screens.html`); sprawdźcie je w Chrome (DevTools → Inspect) przy pierwszych ekranach.

```
export const colors = {
  bg: '#000000',            // tło ekranów
  surface: '#0d0e10',       // kafelki, karty, popupy
  surface2: '#111214',
  surfaceLow: '#0b0c0e',    // wiersze list (HORN), tło rysunku auta, nieaktywny CRUISE
  border: '#34363a',        // ramki przycisków (menu, CANCEL)
  borderSoft: '#2c2e32',    // lewa krawędź kafelków
  segmentOff: '#2a2c30',    // zgaszone segmenty suwaków
  text: '#f2f3f4',          // nagłówki i wartości
  text2: '#e8e9ea',
  muted: '#b9bbbe',         // etykiety (DRIVER, VOLUME)
  muted2: '#9a9da2',        // HELLO KAROL
  muted3: '#8e9095',        // numery na listach (01 BEEP)
  disabled: '#3a3c40',      // zgaszone kreski i elementy
  disabledBg: '#1a1b1e',    // tło nieaktywnego DRIVE (08A)
  dim: '#4b4e52',           // napis i strzałki nieaktywnego DRIVE, ramki na ekranie jazdy
  onAccent: '#111214',      // napis na żółtym przycisku
  accent: '#F0B429',        // żółty: DRIVE, CREATE, zaznaczenie
  accentDim: '#3a2f18',
  accentTint: 'rgba(240,180,41,0.1)', // tło bieżącej pozycji menu
  danger: '#FF4438',        // E-STOP, DELETE, błędy
  dangerDim: '#2a100e',
  info: '#2FC6DE',          // AUTO-STOP, znaczniki w statystykach
};
export const underglowColors = ['#639922', '#2FC6DE', '#F0B429', '#FF4438',
  '#C0308A', '#7A3CE8', '#F2F3F4'];
export const fonts = { heading: 'SairaCondensed_700Bold', mono: 'JetBrainsMono_500Medium',
  monoBold: 'JetBrainsMono_700Bold' };
export const space = { xs: 4, s: 8, m: 14, l: 22, xl: 30 };   // 22 = boczny margines
export const overlay = 'rgba(0,0,0,0.72)';                     // tło pod popupami
```

Lista nie obejmuje rzadkich odcieni z makiet (np. `#7d8085` — MIN/MAX w statystykach, `#6d7075` — obręcz drążka). Dopisujcie je do `colors` przy ekranie, który ich używa. Nieaktywny CREATE w makiecie 11 ma minimalnie inne odcienie niż DRIVE w 08A — wystarczy jeden styl „disabled”.

Skalowanie: makiety pionowe mają szerokość roboczo 372 pt, iPhone 12–14 — 390 pt. W pionie przenosicie wymiary prawie 1:1 i trzymacie margines 22. Ekran poziomy trzeba skalować (sekcja 11) — to zadanie A w Sprincie 2.

Gotowe, gdy:

- [ ] w ekranach i komponentach nie ma kolorów hex — tylko `colors.*`
- [ ] `underglowColors` zgadza się z listą uzgodnioną w W2

B3 — Nawigacja i ekrany-zaślepki

B · 3 h · 5 XI · zależy od: W1

Po co: cała mapa ekranów istnieje od pierwszego tygodnia, więc każde może pracować na swoim ekranie bez czekania.

1. `app/_layout.tsx`: `GestureHandlerRootView` (styl `flex: 1`) → providery (później `ConnectionProvider` i `ProfileProvider`) → `<Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: colors.bg } }} />`.
2. Pliki tras jak w strukturze projektu: `index`, `menu`, `drive`, `profiles`, `drive-settings`, `light-sound`, `radio`, `telemetry`, `dev`. Każda zaślepka: tytuł ekranu i napis TODO.
3. Opcje tras: `menu` z `presentation: 'transparentModal'` i `animation: 'fade'`; `drive` z `gestureEnabled: false`.
4. Orientacja: w `_layout` przy starcie `ScreenOrientation.lockAsync(ScreenOrientation.OrientationLock.PORTRAIT_UP)`.
5. Stos ekranów ma być płytki: z menu zawsze Home → wybrany ekran (np. `router.dismissAll()` i `router.push('/radio')`; sprawdźcie w dokumentacji expo-router, która funkcja robi to w waszej wersji).
6. Jasny pasek stanu: `<StatusBar style="light" />` z `expo-status-bar`.

Gotowe, gdy:

- [ ] z Home przez menu wchodzę na każdy ekran; po 10 przejściach gest „wstecz” wraca od razu do Home
- [ ] aplikacja jest zawsze w pionie, poza ekranem jazdy (A7)

B4 — Baza SQLite i repozytorium profili

B · 5 h · 6–8 XI · zależy od: W1

Po co: profile i historia sesji żyją w SQLite (sekcja 7). Schemat obu tabel ustalacie teraz, żeby w Sprincie 3 A tylko zapisywał sesje.

1. `src/db/db.ts` — jedna baza dla całej aplikacji, dostępna także spoza komponentów (A6 jej potrzebuje):

```
import * as SQLite from 'expo-sqlite';

let dbPromise: Promise<SQLite.SQLiteDatabase> | null = null;

export function getDb() {
  if (!dbPromise) {
    dbPromise = SQLite.openDatabaseAsync('webcar.db').then(async (db) => {
      await migrate(db);
      return db;
    });
  }
  return dbPromise;
}

async function migrate(db: SQLite.SQLiteDatabase) {
  const row = await db.getFirstAsync<{ user_version: number }>('PRAGMA user_version');
  if ((row?.user_version ?? 0) >= 1) return;
  await db.execAsync(SCHEMA_V1);
  await seedDefaultProfile(db);
  await db.execAsync('PRAGMA user_version = 1');
}
```

2. Schemat `SCHEMA_V1` — wartości `DEFAULT` to wartości domyślne z sekcji 7:

```
CREATE TABLE profiles (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE COLLATE NOCASE,   -- 1–12 znaków, unikalna (bez wielkości liter)
  steering TEXT NOT NULL DEFAULT 'STICK',     -- 'STICK' | 'GYRO'
  camera INTEGER NOT NULL DEFAULT 1,          -- 0/1
  blinkers INTEGER NOT NULL DEFAULT 1,        -- 0/1 → cfg.blk
  power_limit INTEGER NOT NULL DEFAULT 30,    -- 0–100 % → cfg.limit
  autostop_cm INTEGER NOT NULL DEFAULT 30,    -- 0–100 cm → cfg.stop
  led_color TEXT NOT NULL DEFAULT '#639922',  -- 1 z 7 kolorów → cfg.led
  brightness INTEGER NOT NULL DEFAULT 50,     -- 0–100 % → cfg.bri
  horn INTEGER NOT NULL DEFAULT 1,            -- 1–6 → cfg.horn
  volume INTEGER NOT NULL DEFAULT 15,         -- 0–30 → cfg.vol
  radio INTEGER NOT NULL DEFAULT 0            -- 0 = OFF, 1–5 → act radio
);
CREATE TABLE app_state (key TEXT PRIMARY KEY, value TEXT NOT NULL);   -- 'active_profile_id'
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  started_at TEXT NOT NULL,          -- ISO; czas nadaje aplikacja (auto nie ma zegara)
  driver_name TEXT NOT NULL,         -- tekst, nie id: sesja zostaje po usunięciu profilu
  duration_s INTEGER NOT NULL,
  closest_front_cm INTEGER, closest_rear_cm INTEGER,   -- NULL = brak pomiaru
  power_min REAL, power_avg REAL, power_max REAL,
  ping_min REAL, ping_avg REAL, ping_max REAL,
  jitter_min REAL, jitter_avg REAL, jitter_max REAL,
  rssi_min REAL, rssi_avg REAL, rssi_max REAL,
  temp_min REAL, temp_avg REAL, temp_max REAL
);
```

3. Profil startowy (`seedDefaultProfile`, sekcja 7): przy pustej bazie jeden profil `DRIVER` z wartościami domyślnymi, ustawiony jako aktywny. Dzięki `DEFAULT` w schemacie wystarczy podać samą nazwę:

```
async function seedDefaultProfile(db: SQLite.SQLiteDatabase) {
  const r = await db.runAsync("INSERT INTO profiles (name) VALUES ('DRIVER')");
  await db.runAsync("INSERT INTO app_state (key, value) VALUES ('active_profile_id', ?)",
    String(r.lastInsertRowId));
}
```

4. `src/db/profiles.ts`:
   - `getProfiles()` — aktywny pierwszy, potem alfabetycznie; `getProfile(id)`;
   - `getActiveProfile()` (id z `app_state`); `setActiveProfile(id)`;
   - `createProfile(name)` → nowe id z polami domyślnymi; `renameProfile(id, name)` — edycja profilu zmienia tylko nazwę (sekcja 7);
   - `updateProfile(id, changes)` — dla ekranów ustawień w Sprincie 2 (autozapis);
   - `deleteProfile(id)` — rzuca błąd dla aktywnego profilu, więc zawsze zostaje co najmniej jeden;
   - `normalizeName(raw)` = `raw.trim()`; `isNameValid(name, profiles, excludeId?)` — 1–12 znaków po obcięciu i brak duplikatu bez względu na wielkość liter (sekcja 7);
   - mapowanie pól z bazy (`power_limit`) na typ `Profile` (`powerLimit`); `camera` i `blinkers` z 0/1 na `boolean`;
   - parametry zawsze przez `?` (`db.runAsync('UPDATE profiles SET name = ? WHERE id = ?', name, id)`), nigdy przez sklejanie tekstu — nazwę wpisuje użytkownik.

5. `resetDb()` do celów deweloperskich: usuwa tabele, ustawia `PRAGMA user_version = 0` i migruje od nowa (przycisk w B9).

Gotowe, gdy:

- [ ] przy pierwszym starcie jest 1 profil DRIVER (STICK, 30 %, radio OFF) i jest aktywny; po zamknięciu i ponownym otwarciu aplikacji dane zostają
- [ ] ręczny test: utworzenie „TEST", zmiana nazwy, usunięcie działają; `isNameValid('driver', …)` zwraca `false`; usunięcie aktywnego rzuca błąd
- [ ] A używa `getActiveProfile()` w A6

B5 — ProfileProvider i useProfiles

B · 2 h · 9 XI · zależy od: B4

1. `src/state/ProfileContext.tsx`: przy starcie ładuje `profiles` i `activeProfile`.
2. Akcje `activate(id)`, `create(name)`, `rename(id, name)`, `remove(id)`, `update(id, changes)` — każda woła repozytorium, a potem `reload()`.
3. W `activate` po zapisie wywołaj `syncActiveProfile()` z A6 (sekcja 7: po aktywacji ustawienia i stacja radia od razu idą do auta). Do czasu A6 — komentarz TODO.
4. W `update`: gdy zmieniasz aktywny profil, po zapisie też `syncActiveProfile()`. Z tego korzystają w Sprincie 2 ekrany Drive settings, Light & sound i Radio — autozapis bez SAVE (sekcja 7).
5. Provider w `app/_layout.tsx`.

Gotowe, gdy:

- [ ] Home czyta `activeProfile` z `useProfiles()`
- [ ] zmiana aktywnego profilu (np. z ekranu `/dev`) odświeża Home bez restartu

B6 — Komponenty bazowe

B · 7 h · 9–11 XI · zależy od: B1, B2

Po co: te same elementy powtarzają się na 25 stanach makiet. Zrobione raz — w Sprincie 2 ekrany składa się z gotowych klocków.

Komponent	Wygląd z makiet	Gdzie
`ScreenHeader({ title })`	żółta kreska 6×46 pochylona o −12°; nad tytułem „WEBCAR" (Label, accent); tytuł Heading 34; po prawej `MenuButton`; padding 30/22/22	10, 13, 14, 15, 17, 18
`MenuButton({ onPress })`	kwadrat 48×48, ramka 1,5 `border`, trzy paski 20×3 `text2` co 5	wszystkie ekrany pionowe
`PrimaryButton({ label, onPress, disabled, size, icon })`	tło accent, napis Heading 44 (DRIVE) albo 28 (pozostałe) w kolorze `onAccent`, po prawej dwie strzałki » (TEST HORN: ikona głośnika), ścięty prawy dolny róg 18 (DRIVE) albo 14; wyłączony: tło `disabledBg`, napis i strzałki `dim` (08A)	DRIVE, CREATE i SAVE (okna profilu), USE THIS PROFILE, TEST HORN, CLEAR HISTORY, START DRIVING, BACK TO DRIVING
`Button({ label, variant })`	wysokość 52, napis Heading 24; `outline`: ramka 1,5 `border`, napis `text2`; `danger`: tło danger, napis prawie czarny, ścięty róg 10 (CLOSE w 19 też ma ścięty róg)	outline: CANCEL, QUIT, CLOSE; danger: DELETE, CLEAR, RETRY

Komponent	Wygląd z makiet	Gdzie
`InfoTile({ label, children })`	tło surface, lewa krawędź 4 `borderSoft`, Label 12, wartość Heading 26	Home (4 kafelki)
`SignalBars({ level, color, size })`	4 paski pochylone o −14°, odstęp 3; Home: szerokość 5, wysokości 9/14/19/24; ekran jazdy: szerokość 6, wysokości 10/16/22/28; zgaszone w kolorze `disabled`	Home, ekran jazdy
`Dialog({ visible, title, marker, children })`	pod spodem `overlay`; panel surface z lewym czerwonym paskiem 3; tytuł Heading 32 (na ekranie jazdy 34); kwadracik 13×13 przed tytułem tylko przy błędach połączenia (06, 19)	06, 11B, 16, 19

Wskazówki:

- React Native nie ma `clip-path`. Ścięty róg zróbcie małym kwadratem w kolorze tła obróconym o 45° w rogu albo trójkątem z `react-native-svg`.
- `Dialog` na bazie `Modal` z `transparent` i `animationType="fade"`. Dodajcie `supportedOrientations={['portrait', 'landscape']}` — bez tego CONNECTION LOST na poziomym ekranie jazdy może się zachować źle.
- Okno formularza z makiet 11 i 11A (czarne tło, ramka 1,5, żółta pochylona kreska, ✕, ścięty róg 18) wygląda inaczej niż `Dialog` — powstaje w Sprincie 2 razem z ekranem profili.
- Na Sprint 2 zostają: suwak segmentowy, przełącznik OFF/ON i STICK/GYRO, wiersz listy, próbnik kolorów, kafelek statystyki, okno formularza.

Gotowe, gdy:

- [ ] każdy komponent jest pokazany obok zrzutu z makiety (w PR albo na `/dev`)
- [ ] komponenty używają tylko `theme.ts` i `Typography`
- [ ] `Dialog` otwiera się i zamyka płynnie, tło jest przyciemnione

B7 — Ekran Menu (09)

B · 2 h · 12 XI · zależy od: B3, B6

1. `app/menu.tsx` jak makieta 09: po lewej przyciemniony pasek poprzedniego ekranu (dotknięcie zamyka menu), po prawej panel z kreską, napisem WEBCAR i przyciskiem ✕. Pod spodem 6 pozycji: HOME, DRIVE SETTINGS, DRIVER PROFILES, RADIO, LIGHT & SOUND, TELEMETRY.
2. Bieżący ekran wyróżniony: tło `accentTint`, napis i lewy pasek accent. Menu dostaje go w parametrze: `router.push({ pathname: '/menu', params: { from: '/radio' } })`.
3. Wyeksportuj listę pozycji jako `MENU_ITEMS` — A użyje jej w Sprincie 2 w menu na ekranie jazdy (07).

Gotowe, gdy:

- [ ] każda z 6 pozycji prowadzi na właściwy ekran; ✕ i dotknięcie tła zamykają menu
- [ ] bieżący ekran jest wyróżniony

B8 — Ekran Home (08, 08A)

B · 3 h · 12–13 XI · zależy od: B5, B6, A5

1. Układ wg makiety 08, od góry: „HELLO ” + nazwa aktywnego profilu (Label 16, `muted2` ; po pierwszym starcie HELLO DRIVER) i `MenuButton` ; kreska 8×74 i „WEBCAR” (Heading 88 — może wychodzić za prawą krawędź, tak jest na makiecie); prostokąt 250 na rysunek auta (na razie pusty, ze ściętym rogiem); siatka 2×2 kafelków; DRIVE na dole.
2. Kafelki: DRIVER = `activeProfile.name` ; CONNECTION = `SignalBars` i ONLINE albo czerwony ✕ i OFFLINE (08A); STEERING = STICK/GYRO; POWER LIMIT = `powerLimit` % w kolorze accent.
3. DRIVE jest `disabled` , gdy `status !== 'online'` ; naciśnięcie → `router.push('/drive')` .
4. Do czasu A5 użyj tymczasowo `{ status: 'offline', bars: 0 }` .
5. Górny margines z `useSafeAreaInsets()` — makiety nie uwzględniają paska stanu ani Dynamic Island.

Gotowe, gdy:

- [ ] wygląd zgodny z 08 i 08A (zrzuty w PR; do porównania z makietą — profile z makiet z B9)
- [ ] symulator włączony / wyłączony → ONLINE / OFFLINE w ciągu ok. 2 s; DRIVE nieaktywny przy OFFLINE
- [ ] żaden napis nie wchodzi pod pasek stanu ani Dynamic Island

B9 — Ekran /dev: katalog komponentów (może poczekać)

B · 1,5 h · 14 XI · zależy od: B6

1. `app/dev.tsx` : wszystkie komponenty z B6 w każdym stanie — np. `PrimaryButton` aktywny i wyłączony, `SignalBars` od 0 do 4 w obu rozmiarach, `Dialog` z kwadracikiem i bez.
2. Przyciski deweloperskie: „Reset bazy” (`resetDb()`), „Aktywuj następny profil” oraz „Profile z makiet” — dodaje profile z makiet i aktywuje KAROL, żeby ekrany dało się porównywać z makietami 1:1:

Profil	Kolor	Limit mocy	Pozostałe pola
KAROL	`#639922`	45 %	STICK, kamera ON, kierunkowskazy ON, auto-stop 20 cm, jasność 60 %, klakson 3, głośność 22, radio 1
MAJA	`#C0308A`	domyślny	domyślne
DEMO / EXPO	`#2FC6DE`	25 %	domyślne
BENCH	`#F2F3F4`	100 %	domyślne

3. Wejście: długie przytrzymanie napisu WEBCAR na Home, tylko gdy `__DEV__` .

Gotowe, gdy:

- [ ] ekran pokazuje wszystkie komponenty, a reset bazy przywraca jeden profil DRIVER
- [ ] „Profile z makiet” daje Home jak na makiecie 08 (KAROL, STICK, 45 %)

Czego potrzebujecie od Wojtasa i Wojtka i kiedy

Aplikacja do końca Sprintu 1 nie potrzebuje auta, ale od Sprintu 2 każda funkcja musi być sprawdzona na prawdziwym ESP32. Uzgodnijcie te terminy na spotkaniu W2.

Najpóźniej	Co firmware ma umieć	Do czego w aplikacji
4 XI	protokół v1 zamrożony (W2)	symulator, typy, cała dalsza praca
12 XI	ESP32 w hotspocie ze stałym IP `172.20.10.13`, `WiFi.setSleep(false)`, serwer `/ws`: `hello`, `tel` (mogą być sztuczne wartości), `pong`	test W3, Home ONLINE/OFFLINE na prawdziwym aucie
22 XI	`drv`: PWM serwa i ESC, failsafe 300 ms (moc 0 i koła prosto)	pierwsza jazda z aplikacji, koła w powietrzu
26 XI	E-STOP z zatrzaskiem i `es` w telemetrii; `act led` i gaszenie taśmy po zamknięciu połączenia albo 2 s bez wiadomości; `cfg`: `limit`, `blk`; kierunkowskazy i światła automatyczne	pełny ekran jazdy (Sprint 2)
29 XI	DFPlayer: `act horn` (dźwięk z `cfg.horn`), `act radio`, `r` w telemetrii, klakson w trybie advert; `cfg`: `vol`, `horn`, `led`, `bri`; zapis ustawień do NVS z opóźnieniem	klakson na ekranie jazdy, ekrany Light & sound i Radio (Sprint 2)
2 XII	kamera Xiao: CameraWebServer, stały IP `172.20.10.14`, `/stream` na porcie 81	podgląd wideo, CAMERA OFF i CAMERA CONNECTION LOST (Sprint 3)
6 XII	czujniki ToF (`d1`, `d2`), auto-stop przy `c` = 1 i flaga `as`, pikanie przy cofaniu	tempomat z auto-stopem, statystyki sesji (Sprint 3)

Kamera jest niezależna od głównego ESP32, więc jeśli Wojtas albo Wojtek ma wolną chwilę, warto ją uruchomić wcześniej — od razu z testem z sekcji 12 (pkt 8): czy WebView zgłasza błąd, gdy strumień zerwie się w trakcie. Symulator z A2 mogą traktować jako wzorzec zachowania serwera.

Ryzyka i co ciąć, gdy zabraknie czasu

Największe ryzyko to opóźniony firmware — dlatego symulator jest w pierwszym tygodniu, a jazda z aplikacji na aucie ma twardy termin 29 XI.

Ryzyko	Skutek	Co robicie
Firmware się spóźnia	integracja dopiero w grudniu, błędy wychodzą na końcu	symulator (A2); test W3 jak najwcześniej; jeśli 29 XI auto nie jedzie z aplikacji — rozmowa całego zespołu o priorytetach
Dwa palce naraz nie działają	ekran jazdy bezużyteczny	prototyp A7 w Sprincie 1; plan B z `Gesture.Manual()` opisany w A7
iOS nie pochyla Saira Condensed	napisy proste, inne niż makiety	test w B1 w pierwszych dniach; obejście `skewX` w jednym komponencie
Adresy hotspotu inne niż `172.20.10.x`	aplikacja nie znajdzie auta	test W3; zmiana to jedna stała w firmware i jedna w `config.ts`

Ryzyko	Skutek	Co robicie
WebView nie zgłasza zerwania strumienia kamery	CAMERA CONNECTION LOST (05A) się nie pokaże, obraz tylko zamarznie	sprawdzić przy uruchomieniu kamery (2 XII); obejście z sekcji 12: co 3 s odpytywać `http://172.20.10.14/status`
Expo wyda nowe SDK, a Expo Go na iPhonie się zaktualizuje	projekt przestaje się otwierać w Expo Go	aktualizacja projektu (`npx expo install expo@latest` , potem `npx expo install --fix`) — zarezerwujcie na to 2 h, gdyby się zdarzyło
Sprint 1 za duży (35 h na osobę)	niedokończone fundamenty	tnijcie w tej kolejności: B9, A8, A6 (przechodzi do Sprintu 2); W3 zależy od firmware

Jeśli w grudniu zabraknie czasu na całą aplikację: wymagania z sekcji 10 (sterowanie, wideo, telemetria, profile, historia sesji, dźwięk i światło) są nienaruszalne. Pierwszymi kandydatami do uproszczenia są tryb GYRO, rysunek auta na Home i dopracowanie animacji.

Zmiany w tej wersji (3 XI)

Dokument dostosowany do kontekstu v2.14 i aktualnych makiet. Najważniejsze zmiany względem wersji z 2 XI:

- **Profil startowy:** przy pierwszym starcie jest jeden profil DRIVER z wartościami domyślnymi z sekcji 7 (limit 30 %, auto-stop 30 cm, jasność 50 %, klakson 1, głośność 15, radio OFF) zamiast czterech profili z makiet. Profile z makiet dodaje przycisk w `/dev` (B9). Zmienione: demo, B4, B8, B9, A6.
- **Ping co 0,5 s na każdym ekranie** (było 1 s). Firmware gasi taśmę po zamknięciu połączenia albo 2 s bez wiadomości — symulator to naśladuje. Zmienione: W2, A1, A2, A4.
- **Synchronizacja po połączeniu wysyła też `act led`** zgodnie z bieżącym ekranem. W kontrakcie doszło `setLed()` , a `syncActiveProfile()` obsługuje radio i autozapis ustawień w Sprincie 2. Zmienione: kontrakt, A6, B5, A7 (uwaga na Sprint 2).
- **`act horn` bez numeru** — zawsze gra dźwięk z `cfg.horn` . Klakson w tabeli firmware przeniesiony do 29 XI, bo gra z DFPlayera. Zmienione: A1, W2, tabela firmware.
- **Ekrany ustawień bez SAVE (autozapis)** i nowy stan 05A (CAMERA CONNECTION LOST). Zmienione: tabele obszarów, harmonogram, B6, tabela firmware, ryzyka.
- **Motyw i komponenty wg aktualnego HTML:** brakujące kolory, styl nieaktywnego przycisku, `Dialog` oddzielony od okna formularza (11, 11A), dwa rozmiary `SignalBars` , wyróżnienie w menu (`accentTint`).
- **Drobne poprawki:** `@types/jest` w A8 (bez niego `tsc` nie zna `expect`), przejście w tło bez ponownego łączenia (A3), `supportedOrientations` z `'landscape'` w `Dialog` , A5 zgodny z kontraktem, symulator z blokadą auto-stopu jak w sekcji 4, imiona Wojtas i Wojtek jak w sekcji 1 kontekstu.